

Logistics

MDPs

Imitation Learnin

DaGGER

# RLRS Week 2: MDPs, DaGGER (IL)

Carrie Chen and the CUAi team

# Logistics

Next week: our first guest speaker ZhongZhu Zhao from Together AI   



## Zhongzhu (Charlie) Zhou

/ZHONG-JOO JOH/

Senior Research Scientist



Ph.D.

School of Computer Science, Faculty of Engineering  
The University of Sydney

B.Eng. (Hons)

School of Computer Science and Engineering  
Sun Yat-sen University



## Aurora (Adaptive Speculator)

Static: 337.0 token/s

Aurora: 336.4 token/s

```
$ decode --mode static-speculator

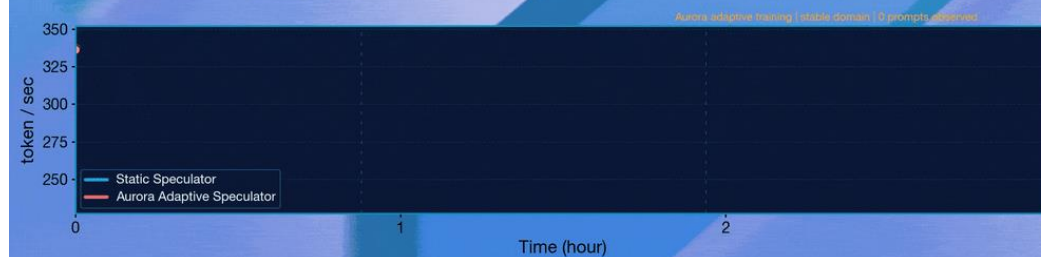
...
malformed tool calls and partial JSON emissions
while preserving decoding throughput. Build a
training feedback loop that updates proposer
heuristics using recent successful draft
trajectories from online data. Refactor request
scheduling to prevent starvation between long-
context prompts and short interactive coding
Generation:
(streaming...)
```

```
$ decode --mode aurora-adaptive

...
users. Build a training feedback loop that updates
proposer heuristics using recent successful draft
trajectories from online data. Optimize tokenizer
and detokenizer pipelines to reduce host CPU
overhead and improve streaming smoothness for
users. Build a training feedback loop that updates
proposer heuristics using recent
Generation:
(streaming...)
```

Static Speculator

Aurora (Adaptive)



1.57x

Speedup on MiniMax M2.1 in mixed-data scenarios.



1.25x

Speedup over static speculators on Qwen3/Llama3.



Cost Efficient

Optimized serving footprint through continuous adaptation.

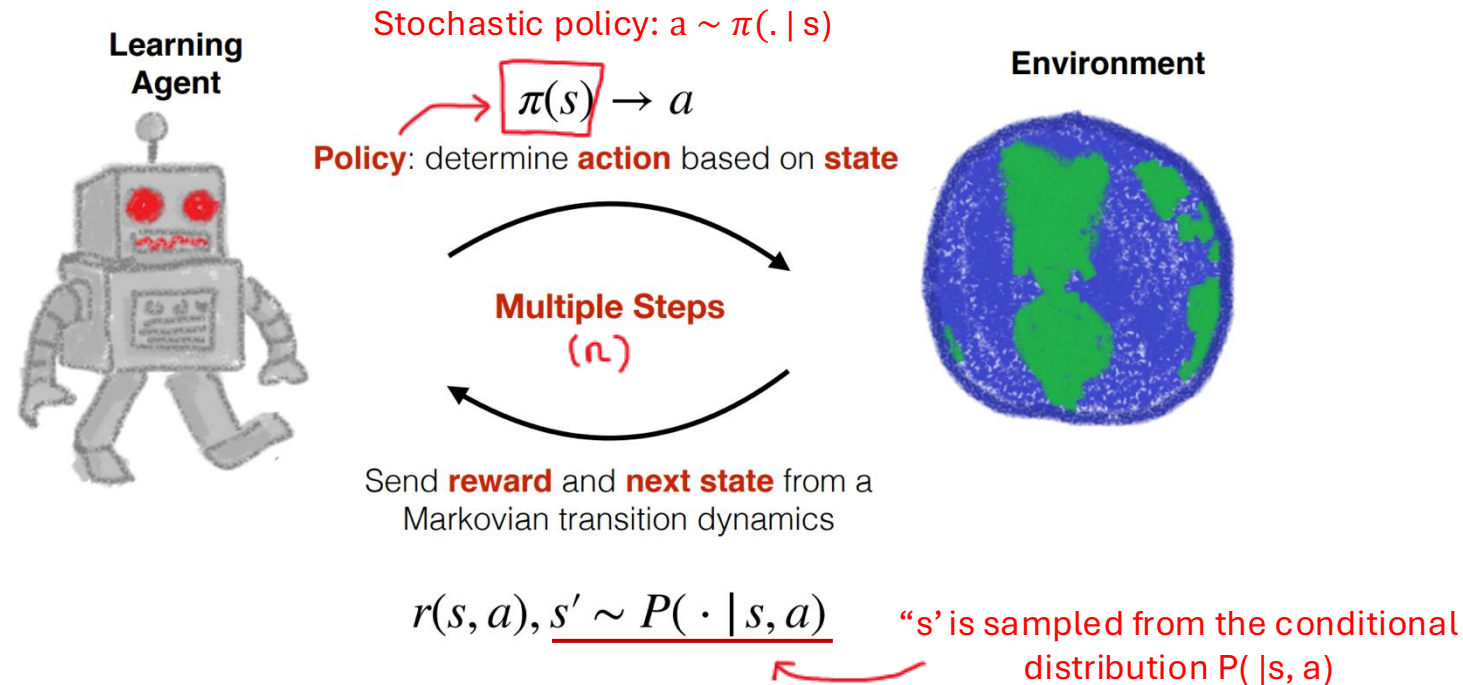


Open Source

Built on SGLang with full implementation available.

# Introducing MDPs (Markov Decision Processes)

Mathematical framework for formalizing sequential decision making



Example: move the ball to (x, y, z) goal state



(states)  $s$ : robot configuration and the ball's position

(actions)  $a$ : Torque on joints in arm & fingers

(transitions)  $P$ :  $s' \sim P(\cdot | s, a)$  physics + noise

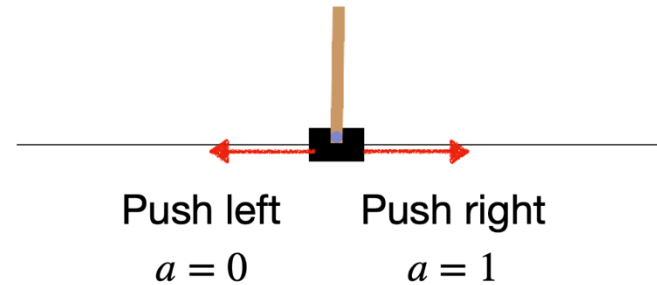
(reward)  $r$ : -distance (goal, ball) – torque magnitude

# What is a policy?

$\pi: S \rightarrow A$ , A mapping between states and actions (can be stochastic/probabilistic, can be deterministic) i.e. what action should I take if I'm in x state?

OpenAI Gym Cartpole Example:

State = [cart pos, cart velocity,  
pole angle, pole angular velocity]



$$r(s, a) = \begin{cases} 1 & \text{pole angle} \in [-12^\circ, 12^\circ], \\ 0 & \text{else} \end{cases}$$

Policy 1: uniform random  
 $\pi(0 | s) = \pi(1 | s) = 0.5, \forall s$

Policy 2: adaptive

$$\pi(s) = \begin{cases} 0(\text{left}) & \text{if pole angle} < 0 \\ 1(\text{right}) & \text{else} \end{cases}$$

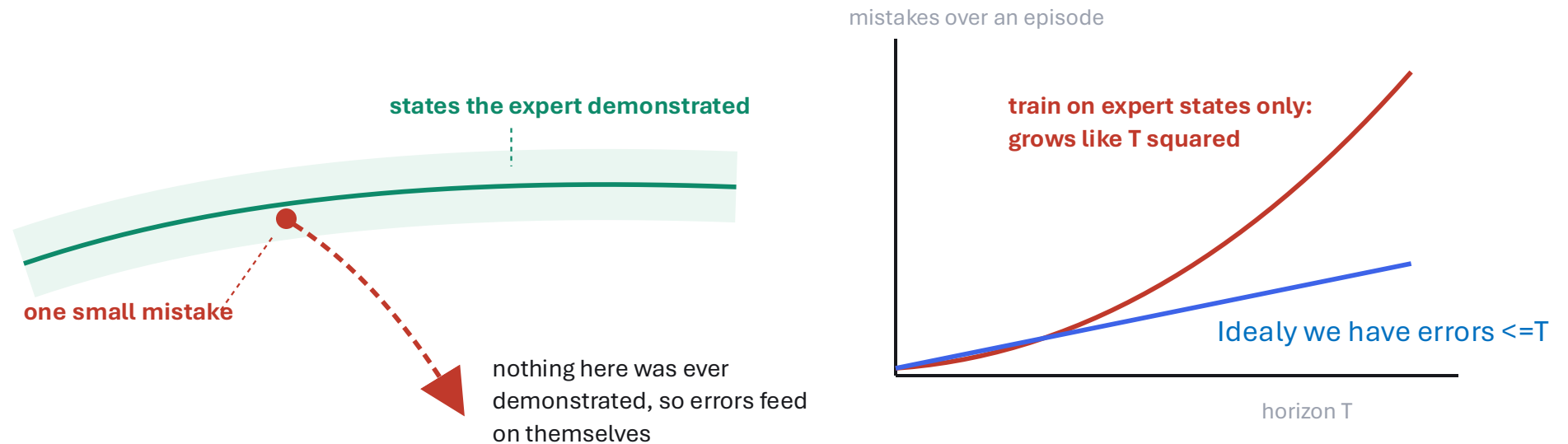
Q: If we have S states and A actions, how many total possible deterministic mappings (policies) are there?

# Defining the problem in a BC setting

Recall the behaviour cloning algorithm: choose regression (for continuous action) or classification loss, and perform SL (supervised learning)

$$\hat{\pi} = \min_{\pi \in \Pi} \sum_{i=1}^M \ell(\pi(s_i^*), a_i^*)$$

What can go wrong? Predictions affect future inputs/ observations!



**Collecting more expert demonstrations does not help**

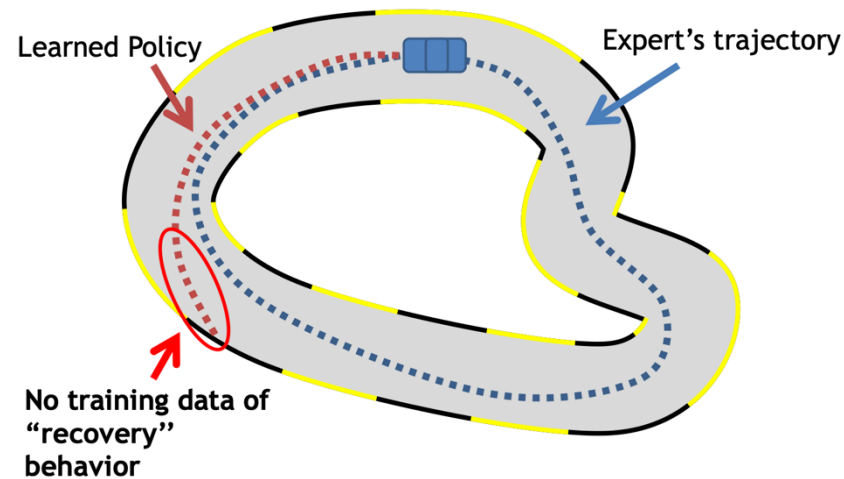
→ The states you're missing coverage in are the ones the expert never encounter

# Defining the problem in a IL setting

Recall the behaviour cloning algorithm: choose regression (for continuous action) or classification loss , and perform SL (supervised learning)

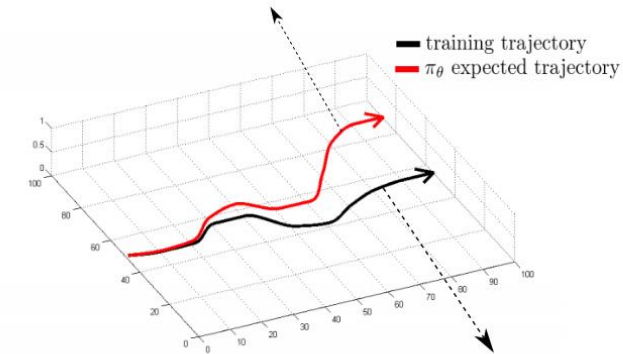
$$\hat{\pi} = \min_{\pi \in \Pi} \sum_{i=1}^M \ell(\pi(s_i^*), a_i^*)$$

What can go wrong? Predictions affect future inputs/ observations!



[Pomerleau89, Daume09]

The trained policy diverges, experiencing states it hasn't seen during training by expert



States visited during expert's interaction with environment. Used for training.

**Collecting more expert demonstrations does not help**

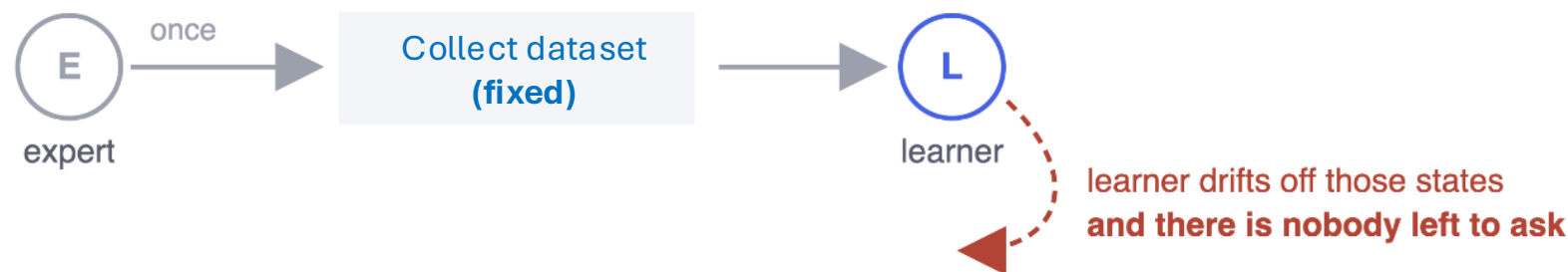
→ The states you're missing coverage in are the ones the expert never encounter

# Defining the problem in a IL setting

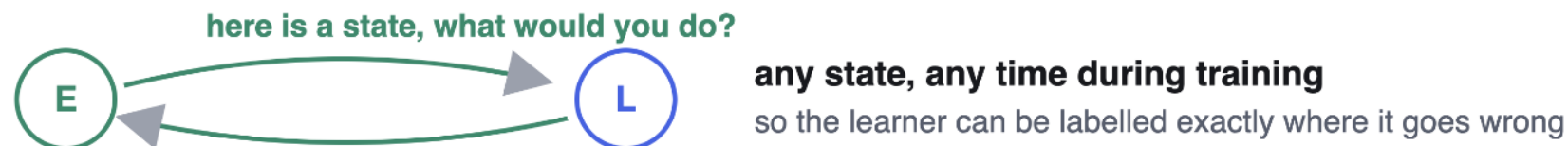
How do we mitigate the distribution shift issue that compounds errors quadratically with respect to the number of time steps?

Assume we can query the expert  $\pi^*$  at each time step:

Offline imitation, what we had



Interactive imitation, what DAgger assumes

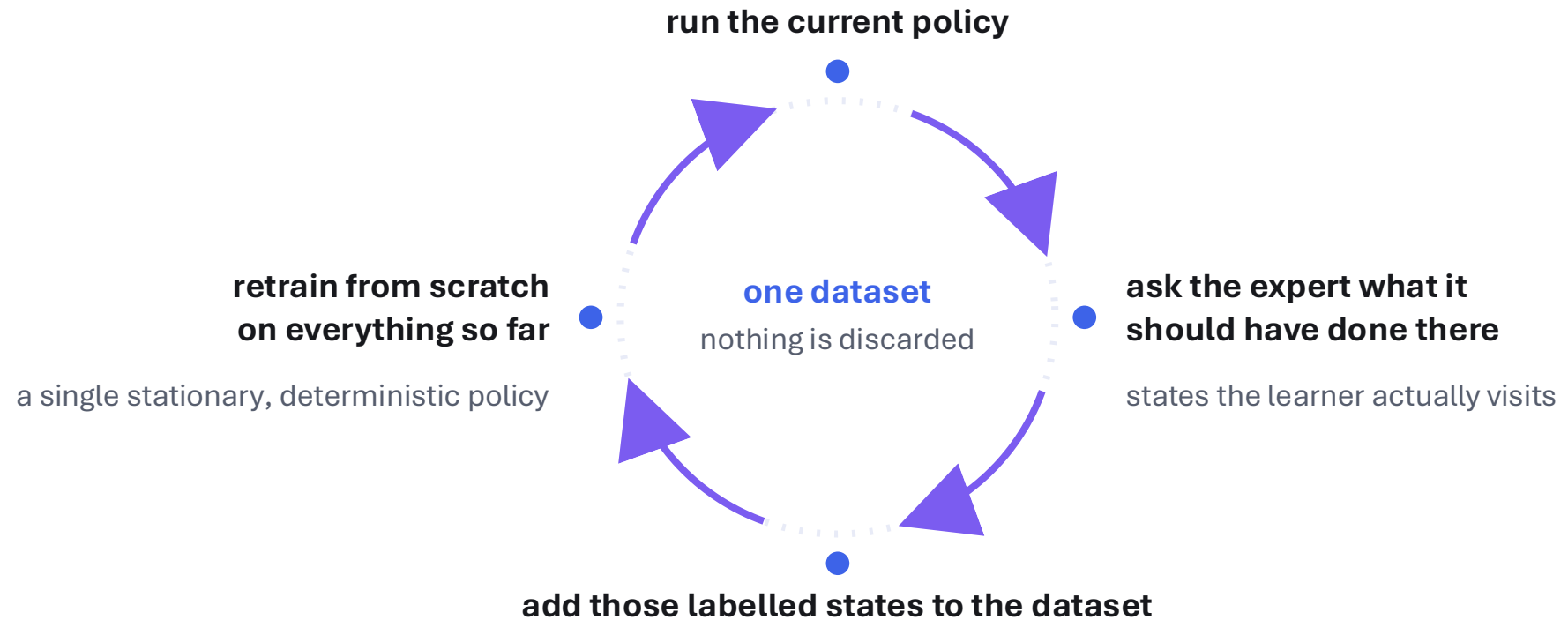


**Collecting more expert demonstrations does not help**

→ The states you're missing coverage in are the ones the expert never encounter

# Dagger's dataset aggregation loop

What if we could query the expert online and iteratively re-train our model?



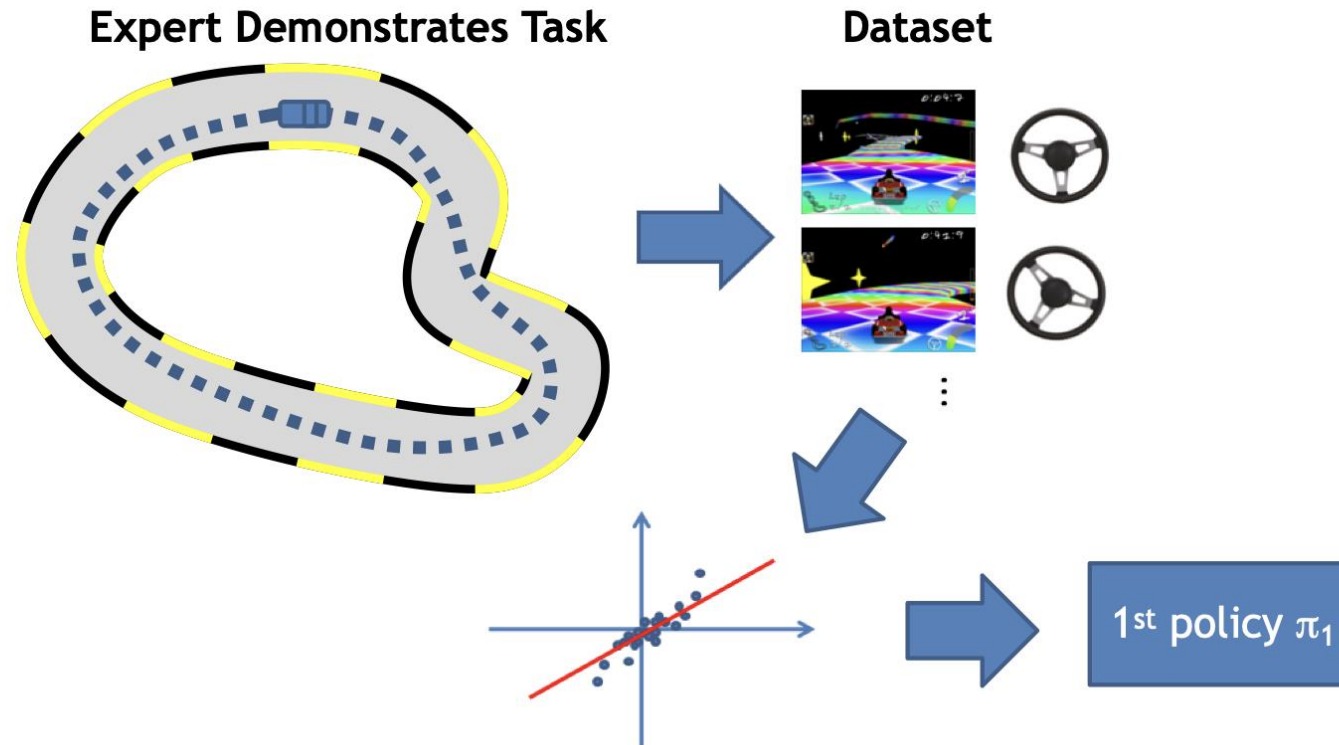
One option: while collecting, let the expert take over a fraction of the actions, decaying over iterations.  
**The parameter free version uses the expert on the first iteration only**, and often works best.



# Dagger's dataset aggregation loop

What if we could query the expert online and iteratively re-train our model?

0th iteration



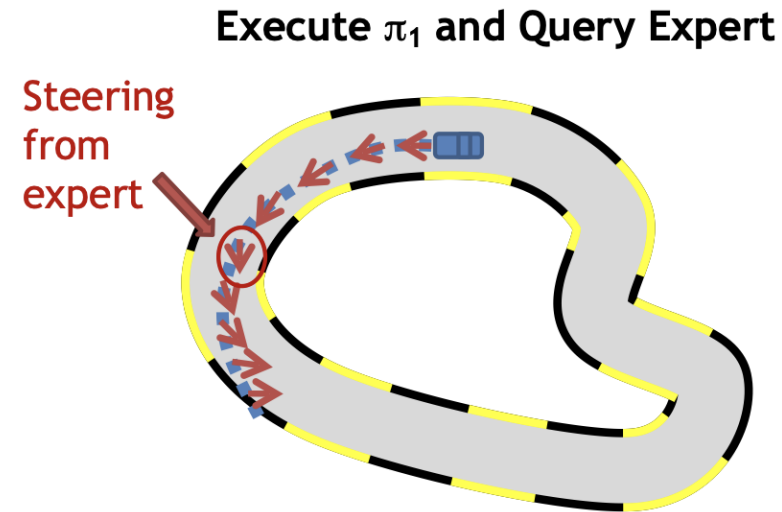
One option: while collecting, let the expert take over a fraction of the actions, decaying over iterations. **The parameter free version uses the expert on the first iteration only**, and often works best.

All DAgger slides credit: Drew Bagnell, Stephane Ross, Arun Venktraman

# Dagger's dataset aggregation loop

What if we could query the expert online and iteratively re-train our model?

## 1st iteration

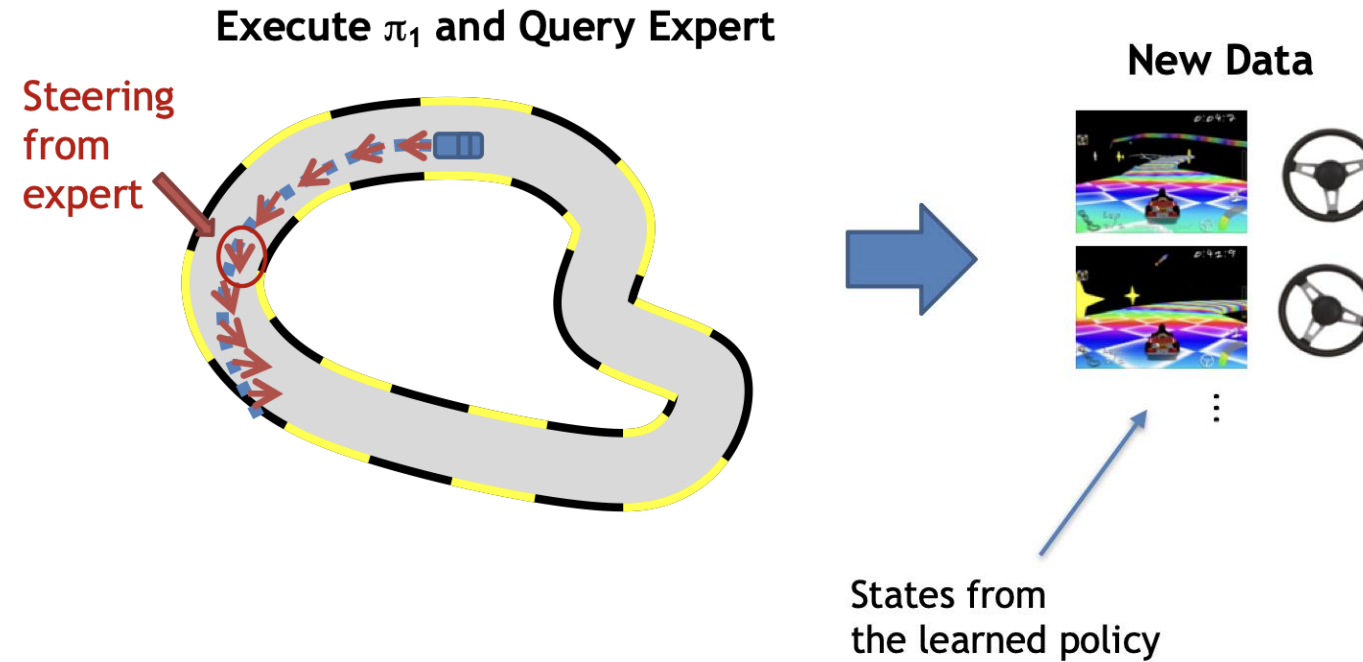


One option: while collecting, let the expert take over a fraction of the actions, decaying over iterations.  
**The parameter free version uses the expert on the first iteration only**, and often works best.

# Dagger's dataset aggregation loop

What if we could query the expert online and iteratively re-train our model?

## 1st iteration

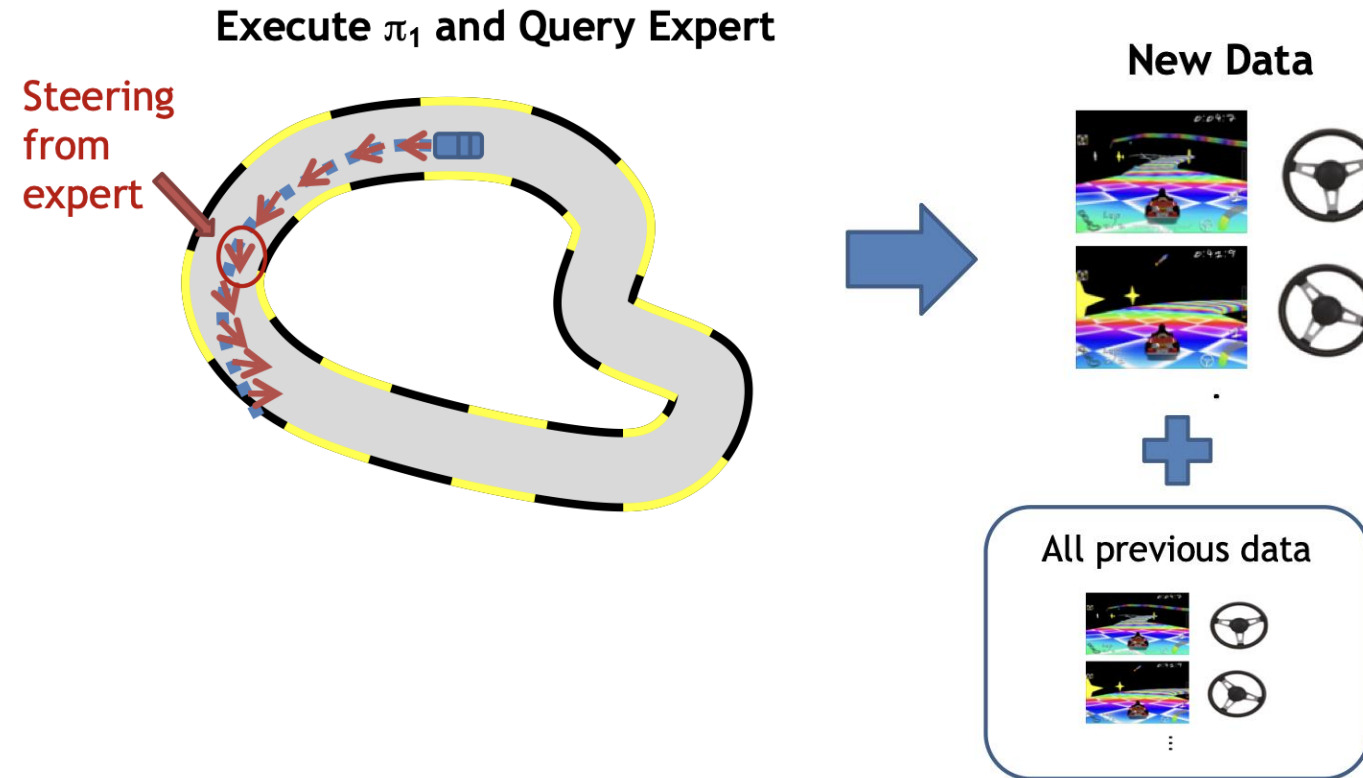


One option: while collecting, let the expert take over a fraction of the actions, decaying over iterations.  
**The parameter free version uses the expert on the first iteration only**, and often works best.

# Dagger's dataset aggregation loop

What if we could query the expert online and iteratively re-train our model?

## 1st iteration

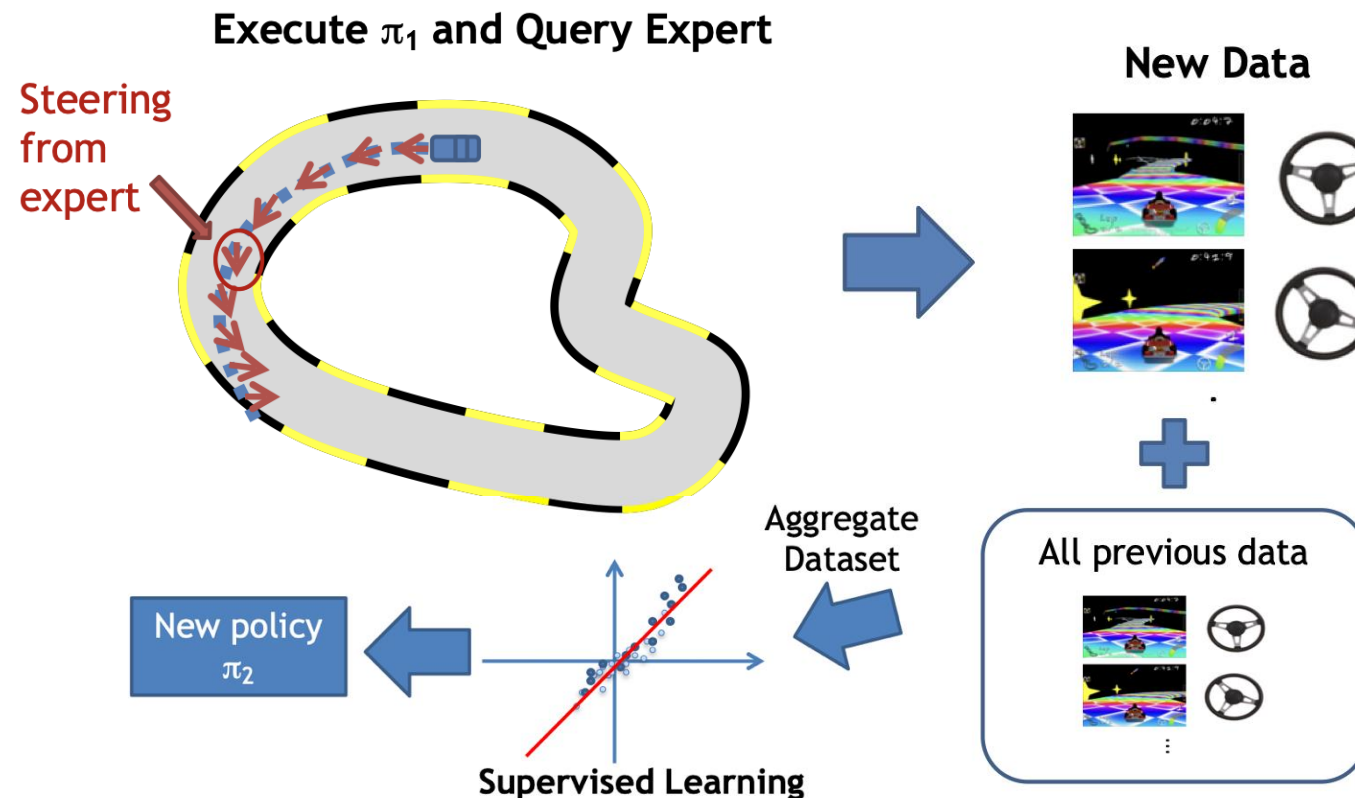


One option: while collecting, let the expert take over a fraction of the actions, decaying over iterations.  
**The parameter free version uses the expert on the first iteration only**, and often works best.

# Dagger's dataset aggregation loop

What if we could query the expert online and iteratively re-train our model?

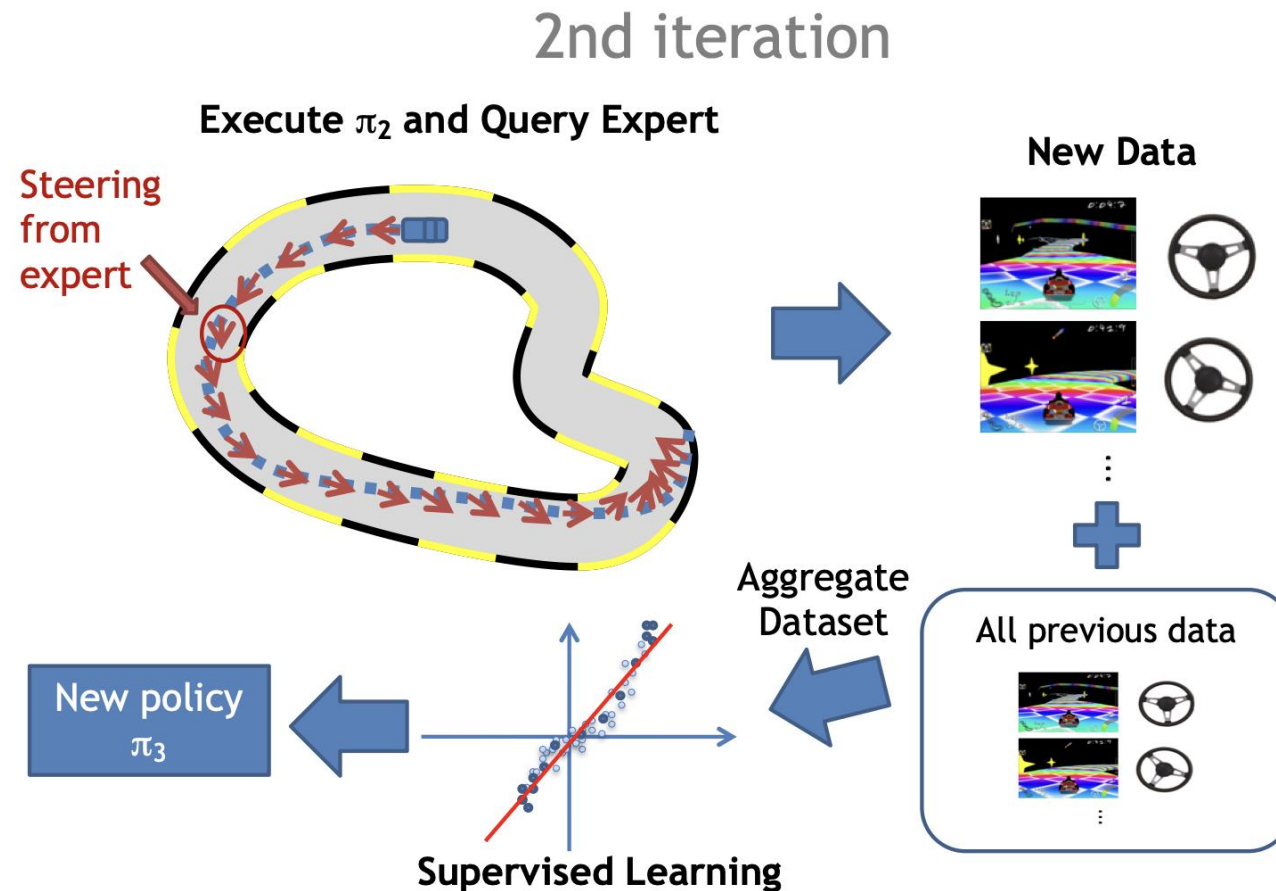
1st iteration



One option: while collecting, let the expert take over a fraction of the actions, decaying over iterations. **The parameter free version uses the expert on the first iteration only**, and often works best.

# Dagger's dataset aggregation loop

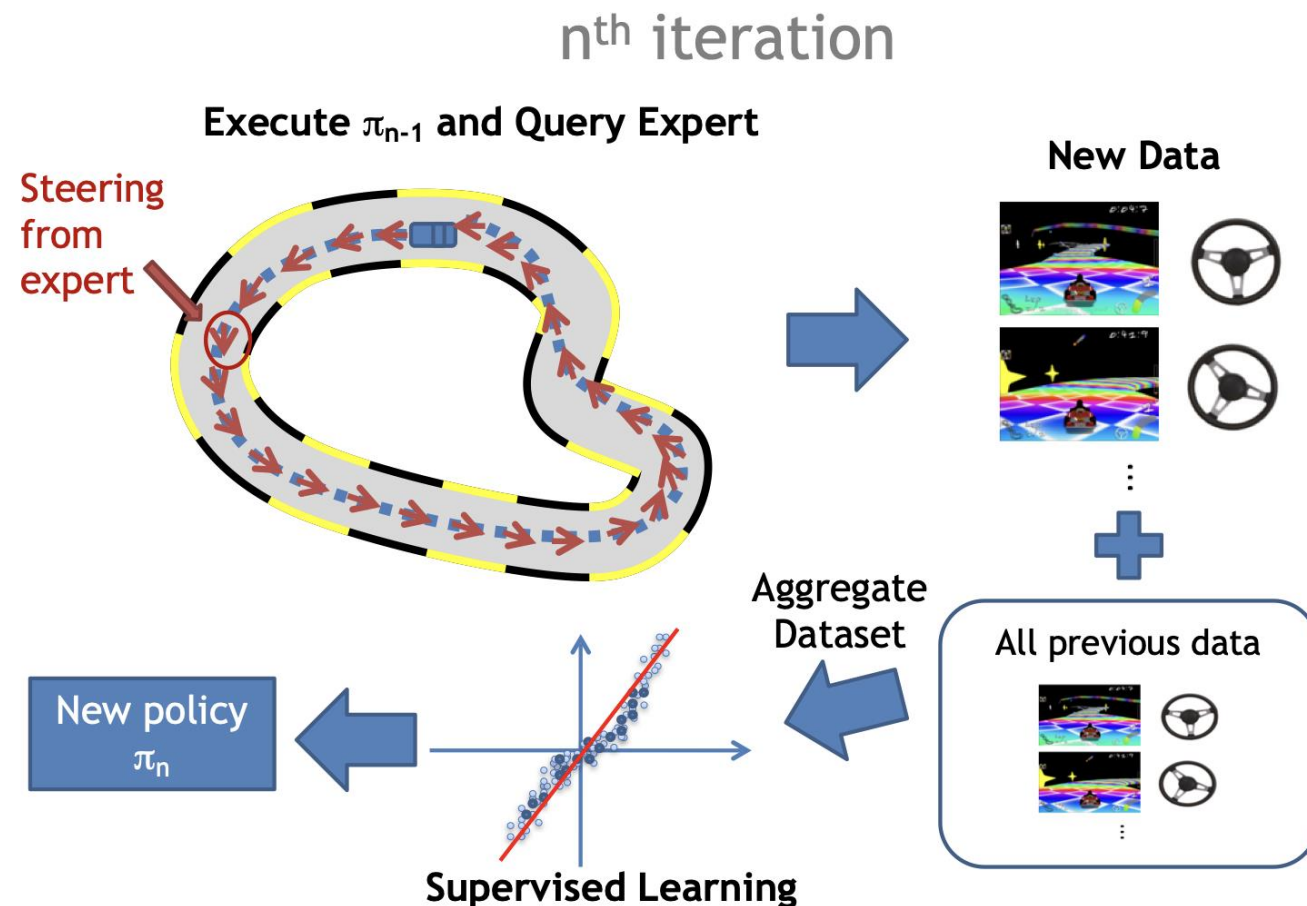
What if we could query the expert online and iteratively re-train our model?



One option: while collecting, let the expert take over a fraction of the actions, decaying over iterations.  
**The parameter free version uses the expert on the first iteration only**, and often works best.

# Dagger's dataset aggregation loop

What if we could query the expert online and iteratively re-train our model?



One option: while collecting, let the expert take over a fraction of the actions, decaying over iterations. **The parameter free version uses the expert on the first iteration only**, and often works best.



# Dagger's dataset aggregation loop

What are forms of experts can we query?

Interactive Expert is expensive, especially when the expert is human... But expert does not have to be human...

## Example: high-speed off-road driving

[Pan et al, RSS 18, Best System Paper]



Fig. 4: The AutoRally car and the test track.

Goal: learn a racing control policy that maps from data on cheap on-board sensors (raw-pixel image) to low-level control (steer and throttle)



→ Steering + throttle

(a) raw image

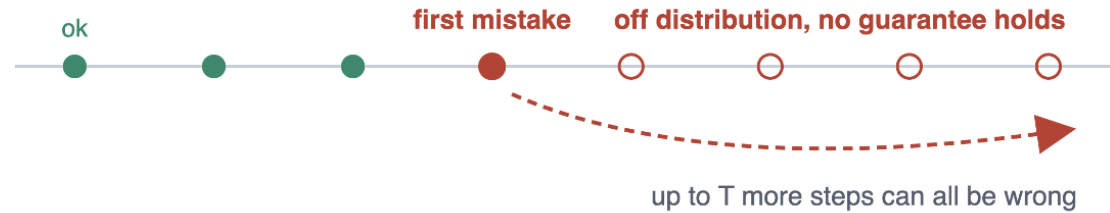
One option: while collecting, let the expert take over a fraction of the actions, decaying over iterations. **The parameter free version uses the expert on the first iteration only**, and often works best.



# Reduction to FTRL in the dataset space

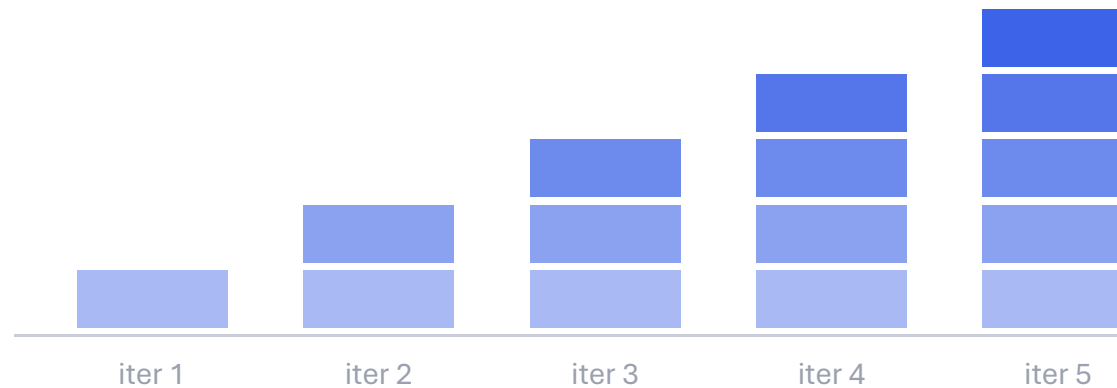
Each iteration plays the policy that is best in hindsight on every state visited so far, does this remind you of a previous algorithm we saw?

a single trajectory of  $T$  steps



the dataset only ever grows

each block is the states one policy visited, labelled by the expert



**Same algorithm, different objects:**

FTRL follows the leader over accumulated gradients. DAgger follows the leader over accumulated datasets

**Aggregation is the stabilizer:**

Keeping old data is what stops the policy from lurching to fit only its newest trajectories, exactly the role curvature plays in FTRL

**The analysis uses no regret:**

Strongly convex losses make Follow the Leader no regret, and any other no regret learner can be dropped in

# Experimental results

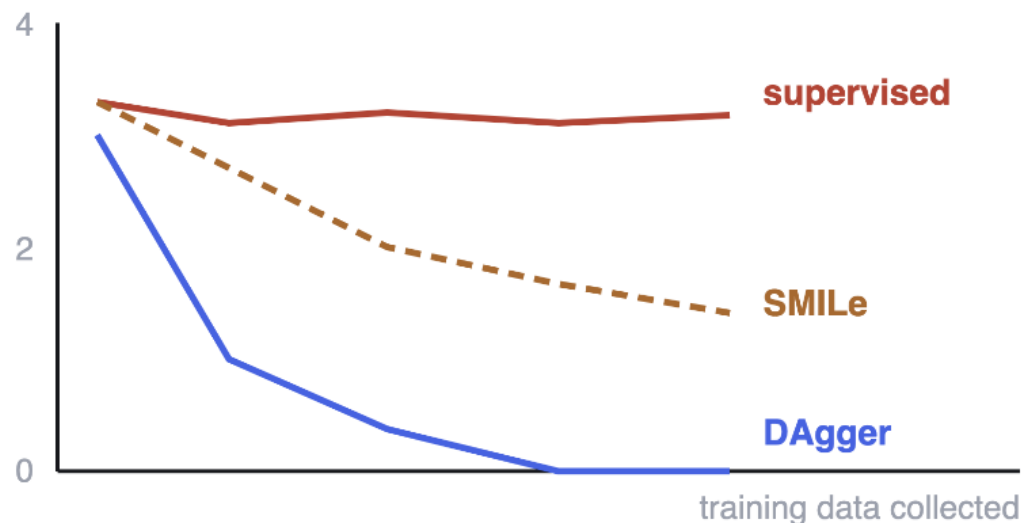
Racing game control, and a handwriting benchmark treated as imitation learning



features  $x$ : LAB color values of each pixel in a 25x19 re-sized image of the 800x600 image  
output steering:  $\hat{y} = w^T x + b$ , where  $w, b$  minimizes ridge regression objective:

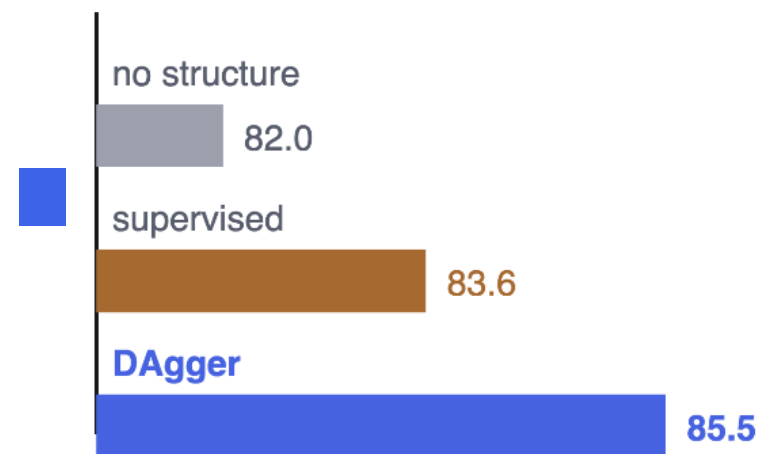
$$L(w, x) = \frac{1}{n} \sum (w^T x_i + b - y_i)^2 + \frac{\lambda}{2} w^T w$$

Super Tux Kart: falls per lap



DAgger: no falls at all after about 15 iterations

Handwriting: character accuracy



bars start at 81 percent so the gaps are visible

On Super Mario Bros, DAgger averaged **3030** distance per stage with a decaying expert mix and **2980** with the parameter free version, ahead of SMILe and every SEARN setting tried.